

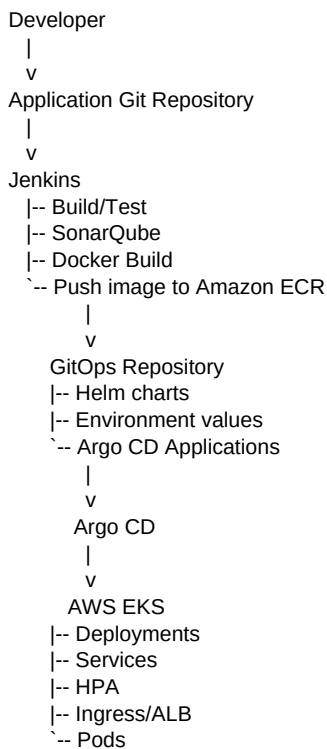
Argo CD + GitOps — 4G Project Interview Revision

Complete production-oriented revision and interview question bank

Project basis: AWS EKS, Kubernetes, Argo CD, Helm, Jenkins, Terraform, Ansible, Prometheus and Grafana. The uploaded CV states that the 4G Microservices Cloud Enablement Platform uses GitOps-based continuous delivery with Argo CD and manages Kubernetes deployments across 76+ worker nodes.

Important: Exact repository paths, sync policies and YAML files were not present in the CV. The repository structure and configuration examples in this guide are production-grade reference examples for interview preparation, not claims about the exact implementation.

1. Reference Architecture



2. Core Concepts

GitOps: Git is the declarative source of truth for the desired application state.

Argo CD: A Kubernetes GitOps continuous-delivery and reconciliation platform.

Desired state: The configuration declared in Git.

Live state: The actual state running in Kubernetes.

Reconciliation: Continuously attempting to make live state match desired state.

OutOfSync: Desired state and live state differ.

Synced: Desired state and live state match.

Self-healing: Argo CD restores drift toward the Git-defined state when enabled.

Pruning: Argo CD removes resources that were deleted from the desired state when enabled.

Refresh: Argo CD checks Git/live state and calculates differences.

Sync: Argo CD applies the desired state to the destination cluster.

3. Production Repository Reference

```

4g-application/
|-- src/
|-- Dockerfile
|-- Jenkinsfile
|-- pom.xml
`-- README.md

4g-gitops/
|-- apps/
|   |-- backend/
|   |   |-- Chart.yaml
|   |   |-- values.yaml
|   |   `-- templates/
|   |       |-- deployment.yaml
|   |       |-- service.yaml
|   |       |-- configmap.yaml
|   |       |-- hpa.yaml
|   |       `-- ingress.yaml
|   |-- frontend/
|   `-- gateway/
|-- environments/
|   |-- dev/
|   |-- staging/
|   `-- prod/
`-- argocd/
    |-- projects/
    |-- applications/
    `-- appsets/

```

4. Example Argo CD Application

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: 4g-backend-prod
spec:
  project: default
  source:
    repoURL: https://github.com/company/4g-gitops.git
    targetRevision: main
    path: apps/backend
  destination:
    server: https://kubernetes.default.svc
    namespace: backend
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```

5. Example Production Helm Values

```

replicaCount: 5

image:
  repository: 123456789.dkr.ecr.ap-south-1.amazonaws.com/4g-backend
  tag: "1.4.27"
  pullPolicy: IfNotPresent

resources:
  requests:
    cpu: 500m
    memory: 512Mi
  limits:
    cpu: "1"
    memory: 1Gi

service:
  type: ClusterIP
  port: 8080

autoscaling:
  enabled: true
  minReplicas: 5

```

maxReplicas: 20
targetCPUUtilizationPercentage: 70

6. Complete Deployment Flow

1. Developer pushes application code.
2. Jenkins checks out source.
3. Jenkins builds and tests.
4. SonarQube/security gates run.
5. Docker image is built.
6. Image is pushed to Amazon ECR with an immutable version.
7. GitOps repository is updated with the image version.
8. Argo CD detects the Git change.
9. Argo CD compares desired state with live EKS state.
10. Argo CD synchronizes the application.
11. Kubernetes performs the rollout.
12. Argo CD reports sync and health status.

Section A — GitOps Fundamentals

Q1. What is GitOps?

Answer: GitOps is an operating model where Git stores the declarative desired state and an automated reconciler applies and maintains that state in the target environment.

Q2. Why do you use GitOps?

Answer: It gives us version-controlled configuration, repeatable deployments, auditability, drift detection and a clean rollback mechanism through Git history.

Q3. What is the source of truth?

Answer: The GitOps repository containing the desired deployment configuration.

Q4. What is desired state?

Answer: The state declared in Git, such as the image version, replicas, resources and Kubernetes configuration.

Q5. What is live state?

Answer: The actual state currently present in the Kubernetes cluster.

Q6. What is reconciliation?

Answer: The process of comparing desired and live state and taking action to make them converge.

Q7. What is drift?

Answer: A difference between Git-defined desired state and the actual Kubernetes state.

Q8. What is OutOfSync?

Answer: Argo CD has detected that desired state and live state differ.

Q9. What is Synced?

Answer: The desired and live states match.

Q10. What are the major GitOps benefits?

Answer: Consistency, traceability, auditability, automated reconciliation, drift correction and controlled rollback.

Section B — Argo CD Architecture and Components

Q1. What is Argo CD?

Answer: Argo CD is a Kubernetes-native GitOps continuous delivery tool that continuously compares Git-defined desired state with live cluster state.

Q2. What is the Argo CD API server?

Answer: It provides the API, UI and CLI-facing control plane for authentication, authorization and application operations.

Q3. What does the repository server do?

Answer: It accesses source repositories and generates manifests from sources such as plain YAML, Helm and Kustomize.

Q4. What does the application controller do?

Answer: It monitors Applications, compares desired and live state, determines sync/health status and performs reconciliation.

Q5. Why is Redis used?

Answer: Argo CD uses Redis for caching and performance-related internal state.

Q6. What is the Argo CD Application resource?

Answer: A custom resource that describes the source repository/path/revision and the destination cluster/namespace plus sync policy.

Q7. What is an AppProject?

Answer: A governance boundary that can restrict repositories, destinations and resource types and can define project-level permissions.

Q8. What is ApplicationSet?

Answer: A controller mechanism for generating and managing many Argo CD Applications from templates and generators.

Q9. What is App of Apps?

Answer: A pattern where one root Application manages other Argo CD Application definitions.

Q10. Can Argo CD manage multiple clusters?

Answer: Yes. Argo CD can manage Applications targeting registered Kubernetes clusters, subject to access and project restrictions.

Section C — Sync, Refresh, Health and Reconciliation

Q1. What is refresh?

Answer: Refresh recalculates the repository and live state comparison so Argo CD can determine whether an Application is synchronized.

Q2. What is sync?

Answer: Sync applies the desired manifests to the target Kubernetes cluster.

Q3. What is automated sync?

Answer: Argo CD automatically synchronizes when it detects an applicable desired-state change.

Q4. What is selfHeal?

Answer: It enables automatic correction when the live state is changed away from the desired state.

Q5. What is prune?

Answer: It allows Argo CD to delete resources that are no longer part of the desired application state.

Q6. Should production always use auto-sync?

Answer: No. The choice depends on change-control, compliance and release strategy. Lower environments often use automated synchronization; production may use controlled or approval-based promotion.

Q7. What is health status?

Answer: It describes the operational health of resources, for example Healthy, Progressing, Degraded, Missing or Unknown.

Q8. Can an Application be Synced but Degraded?

Answer: Yes. Git can match the cluster while the deployed workload is unhealthy, such as Pods failing readiness checks.

Q9. How does Argo CD detect manual drift?

Answer: It compares the live Kubernetes resource state with the desired state generated from Git.

Q10. Why is reconciliation central to GitOps?

Answer: It prevents configuration drift from becoming the long-term state and continuously drives the cluster toward the declared configuration.

Section D — Helm + Argo CD

Q1. Why use Helm with Argo CD?

Answer: Helm provides reusable templates and parameterization; Argo CD manages GitOps reconciliation and application lifecycle.

Q2. Is Helm a replacement for Argo CD?

Answer: No. Helm packages/templates Kubernetes resources; Argo CD provides GitOps delivery and reconciliation.

Q3. How does Argo CD use Helm?

Answer: Argo CD renders the Helm chart using its configured values and reconciles the resulting Kubernetes manifests.

Q4. Why use environment-specific values?

Answer: The same chart can be reused while environment-specific replicas, image tags, resources and other settings vary.

Q5. Where should the production image tag be stored?

Answer: In the GitOps configuration, typically as a Helm value or another declarative configuration field.

Q6. Why avoid latest in production?

Answer: Immutable version tags improve traceability, reproducibility and rollback.

Q7. What happens if Helm rendering fails?

Answer: Argo CD cannot generate valid manifests, so synchronization cannot proceed. Investigate chart syntax, values, dependencies and template logic.

Q8. What should be common across environments?

Answer: The reusable chart/template structure. Environment-specific behavior should be expressed through controlled values or overlays.

Section E — Jenkins + Argo CD Integration

Q1. What is Jenkins responsible for?

Answer: In this architecture Jenkins handles CI: source checkout, build, tests, quality/security checks, container build and image publication.

Q2. What is Argo CD responsible for?

Answer: Argo CD handles GitOps CD: it watches desired deployment configuration, reconciles it and deploys it to EKS.

Q3. Does Argo CD build Docker images?

Answer: No. Image building belongs to CI tooling such as Jenkins.

Q4. Should Jenkins directly run kubectl in a GitOps model?

Answer: Preferably not for the managed production path. Jenkins should update the GitOps desired state and Argo CD should reconcile the cluster.

Q5. What happens after Jenkins pushes an image to ECR?

Answer: The GitOps configuration is updated to reference the new immutable image version; Argo CD detects that desired-state change and synchronizes EKS.

Q6. Why separate CI and CD?

Answer: It reduces CI-to-cluster coupling and establishes a clear boundary: CI creates artifacts; GitOps CD reconciles deployment state.

Q7. What if Jenkins succeeds but deployment fails?

Answer: Check the GitOps commit, Argo CD Application state, sync result, rendered manifests, Kubernetes rollout and workload health separately.

Section F — Multi-Environment and Production Architecture

Q1. How would you structure dev, staging and prod?

Answer: Use reusable charts/templates with environment-specific values or overlays and separate Argo CD Applications or ApplicationSets.

Q2. How does promotion work?

Answer: A tested image version is promoted by changing the desired image version in the target environment's Git configuration.

Q3. Should application source and deployment configuration be separated?

Answer: A common production pattern is yes: application code/CI in one repository and declarative deployment configuration in a GitOps repository.

Q4. How would you protect production?

Answer: Use branch protection, pull-request approvals, Argo CD RBAC, AppProject restrictions, least-privilege Kubernetes access and controlled promotion.

Q5. Why use separate production clusters?

Answer: Where required, separate clusters provide stronger security and blast-radius isolation than sharing a single cluster.

Q6. How can ApplicationSet help a large microservice platform?

Answer: It can generate many Applications consistently from a template, reducing repetitive Application definitions.

Q7. How do you standardize dozens of services?

Answer: Use reusable Helm charts, common values conventions, ApplicationSets and consistent GitOps repository structure.

Q8. What is a good production image strategy?

Answer: Build immutable, uniquely identifiable images and reference the exact version or digest from Git.

Section G — Security, RBAC and Secrets

Q1. How should secrets be managed?

Answer: Do not store plaintext production secrets in Git. Use a secret-management system such as AWS Secrets Manager together with an appropriate Kubernetes integration.

Q2. What is least privilege?

Answer: Grant only the permissions required for a user, service account or component to perform its function.

Q3. How do you restrict Argo CD Applications?

Answer: Use AppProjects to constrain allowed repositories, destinations and resource types, combined with Argo CD RBAC.

Q4. Why is the pull model useful for security?

Answer: CI does not need broad direct production-cluster credentials when Argo CD performs reconciliation from inside the environment.

Q5. How do you prevent unauthorized production changes?

Answer: Use protected Git branches/PR approvals, restricted Argo CD access, Kubernetes RBAC and auditable promotion workflows.

Q6. Should developers have unrestricted production kubectl access?

Answer: Normally no. Access should follow least privilege and the organization's operational model.

Q7. How do you audit changes?

Answer: Git history, pull requests, Argo CD application history and cluster audit mechanisms provide traceability.

Section H — Troubleshooting and Production Scenarios

Q1. Argo CD is OutOfSync. What do you do?

Answer: Inspect the Application diff, identify the resource, determine whether Git or a manual/controller change caused the difference, validate Git configuration and then sync or correct Git as appropriate.

Q2. Jenkins succeeded but Argo CD did not deploy. What do you check?

Answer: Confirm the GitOps repository was updated, branch/path are correct, Argo refreshed, the Application sees the change and whether synchronization failed.

Q3. Argo CD says Synced but Pods are failing. What do you check?

Answer: Check Application health, Deployment, ReplicaSet, Pod status/events, logs, probes, configuration, dependencies and resource constraints.

Q4. New image is in ECR but Pods still run the old image. What do you check?

Answer: Check the GitOps image tag, Helm values, rendered manifest, Argo Application, Deployment rollout and actual Pod image.

Q5. Deployment is stuck in Progressing. What are common causes?

Answer: Image pull problems, insufficient capacity, scheduling failures, readiness probe failures, startup errors or CrashLoopBackOff.

Q6. Pods show ImagePullBackOff. What do you check?

Answer: Image repository/tag, ECR access, workload/node identity, authentication, networking and Pod events.

Q7. Users receive 503 after a successful rollout. What do you check?

Answer: Pod readiness, Service selectors/endpoints, Ingress/ALB routing, target health, networking/security controls and application behavior.

Q8. Someone manually scales a deployment. What happens with selfHeal?

Answer: Argo CD detects the drift and, when configured to self-heal, reconciles the deployment back toward the Git-defined replica count.

Q9. What if Argo CD goes down?

Answer: Existing Kubernetes workloads generally continue running. New Git changes and drift reconciliation are temporarily unavailable until Argo CD recovers.

Q10. What if GitHub is temporarily unavailable?

Answer: Existing workloads continue running. Argo CD cannot retrieve new desired-state changes until repository connectivity is restored.

Section I — Advanced Argo CD

Q1. What is the pull model?

Answer: The cluster-side GitOps controller observes Git and pulls desired state instead of the CI server pushing deployment commands into the cluster.

Q2. Why is pull-based delivery useful?

Answer: It improves separation of duties, can reduce production credentials in CI and makes Git the explicit desired-state source.

Q3. What is a rollback in GitOps?

Answer: Restore a known-good desired state, preferably by reverting the Git configuration/commit so the repository and cluster remain aligned.

Q4. Why is Git revert preferable to an undocumented manual rollback?

Answer: It preserves the durable desired state and creates an auditable history of the rollback.

Q5. Can Argo CD deploy Helm, Kustomize and plain YAML?

Answer: Yes. Argo CD supports multiple manifest-generation approaches, including Helm, Kustomize and plain Kubernetes manifests.

Q6. How do you handle many clusters?

Answer: Register the target clusters, organize Applications with AppProjects/ApplicationSets and enforce appropriate access boundaries.

Q7. How do you troubleshoot an Argo CD sync failure?

Answer: Inspect sync operation details and resource errors, then validate generated manifests, Kubernetes API validation, RBAC, admission policies, quotas and dependencies.

Q8. What is a resource health issue versus sync issue?

Answer: Sync concerns whether desired configuration was applied/matches live state; health concerns whether the resulting resources are operationally healthy.

Q9. What is the difference between Helm templating and reconciliation?

Answer: Helm renders configuration into Kubernetes manifests; reconciliation compares and drives actual cluster state toward those manifests.

Q10. What production principles matter most?

Answer: Declarative Git state, immutable artifacts, least privilege, environment isolation, automated reconciliation, controlled promotion, observable rollouts and auditable rollback.

7. Project-Specific Interview Answers

Explain how Argo CD is used in your 4G project.

In our 4G microservices platform running on AWS EKS, we use Argo CD to implement GitOps-based continuous delivery. Jenkins handles CI activities such as source checkout, build, code-quality checks, Docker image creation and publishing to Amazon ECR. The deployment configuration is maintained in Git, including the image version. Argo CD monitors that desired state, compares it with the live EKS state and synchronizes the application. Helm is used for reusable Kubernetes deployment templates and environment-specific configuration. This improves consistency, traceability, drift detection and rollback.

Explain your complete deployment architecture.

Developer pushes application code to GitHub. Jenkins builds and validates it, runs quality/security checks, creates the container image and pushes it to ECR. The deployment configuration in the GitOps repository is updated with the new image version. Argo CD detects the desired-state change, renders the Helm configuration, compares it with the EKS live state and synchronizes the required Kubernetes resources. Kubernetes performs the rollout, and Argo CD reports synchronization and health status.

Why not Jenkins + kubectl?

Direct Jenkins-to-cluster deployment couples CI to the production cluster and makes the deployment state less Git-centric. With GitOps, Jenkins produces the artifact and updates desired state, while Argo CD performs reconciliation. This provides a stronger audit trail and drift management.

How do you troubleshoot production deployment failure?

I separate the problem into CI, artifact, GitOps and Kubernetes layers: verify Jenkins, verify the ECR image, verify the GitOps commit and Helm values, inspect Argo CD sync/diff/health, then inspect Kubernetes Deployment, ReplicaSet, Pods, events, logs, probes, Services and Ingress/ALB.

How do you perform rollback?

First identify the last known-good version. In a GitOps workflow, revert the GitOps configuration to that version so the desired state is durable and auditable. Argo CD detects the change and reconciles the cluster back to the known-good version.

8. Rapid-Fire Revision Sheet

Term	Interview meaning
Jenkins	CI / artifact creation
ECR	Container image registry
GitOps repo	Desired deployment state
Argo CD	GitOps CD + reconciliation
EKS	Kubernetes runtime
Helm	Templating/package mechanism
Application	Git source + destination + sync configuration
AppProject	Argo governance boundary
ApplicationSet	Application generator
OutOfSync	Desired != live
Synced	Desired == live
Healthy	Workload operational state is healthy
SelfHeal	Correct live drift
Prune	Delete resources removed from desired state

Term	Interview meaning
Rollback	Restore known-good desired state

Final Interview Rule

Memorize the responsibility boundary: Jenkins builds and publishes the artifact → GitOps repository declares what should run → Argo CD reconciles that desired state → EKS runs the workload. If you can explain that flow and troubleshoot each boundary, you can handle most Argo CD interview follow-ups.